

Lab Assignment No.	02
Title	Build a Multiclass classifier using the CNN model. Use MNIST or any other suitable dataset. a. Perform Data Pre-processing b. Define Model and perform training c. Evaluate Results using confusion matrix.
Roll No.	
Class	BE
Date of Completion	
Subject	Computer Laboratory-IV: Deep Learning
Assessor's Sign	

ASSIGNMENT No: 02

Title:- Build a Multiclass classifier using the CNN model. Use MNIST or any other suitable dataset.

- a) Perform Data Pre-processing
- b) Define Model and perform training
- c) Evaluate Results using confusion matrix.

Objective:

Building a multiclass classifier using a Convolutional Neural Network (CNN) using MNIST or any other suitable dataset. It involves several steps, including data pre-processing, defining the model architecture, training the model, and evaluating its performance using a confusion matrix.

Outcome:

Implement the technique of Convolution neural network (CNN)

Software Requirements:

1. Python (3.x recommended)
2. TensorFlow (Deep learning framework for building CNNs)
3. Jupyter Notebook, any Python IDE, or Google Colab (for running Python code)

Hardware Requirements:

1. A machine with at least 8GB of RAM is recommended for model training.
2. A multi-core CPU is suitable, and for faster training, a GPU (Graphics Processing Unit) is highly recommended

Prerequisites:

- Basic understanding of Python programming
- Familiarity with the concepts of Neural Networks, especially Convolutional Neural Networks (CNNs)

Dataset:

<https://github.com/AmritK10/MNIST-CNN>



Libraries or Module used:

- Numpy - for linear algebra.
- Pandas - for data analysis.
- Matplotlib - for data visualization.
- Tensorflow - for neural networks.

Theory:

ANN or Artificial Neural Network is a multi-layer fully-connected neural net that consists of many layers, including an input layer, multiple hidden layers, and an output layer. This is a very popular deep learning algorithm used in various classification tasks like audio and words

Similarly, we have Convolutional Neural Networks (CNNs) for image classification.

CNN is basically a model known to be **Convolutional Neural Network** and in recent times it has gained a lot of popularity because of its usefulness. CNN uses multilayer perceptrons to do computational works. CNN uses relatively little pre-processing compared to other image classification algorithms. This means the network learns through filters that in traditional algorithms were hand-engineered. So, for the image processing tasks CNNs are the best- suited option.

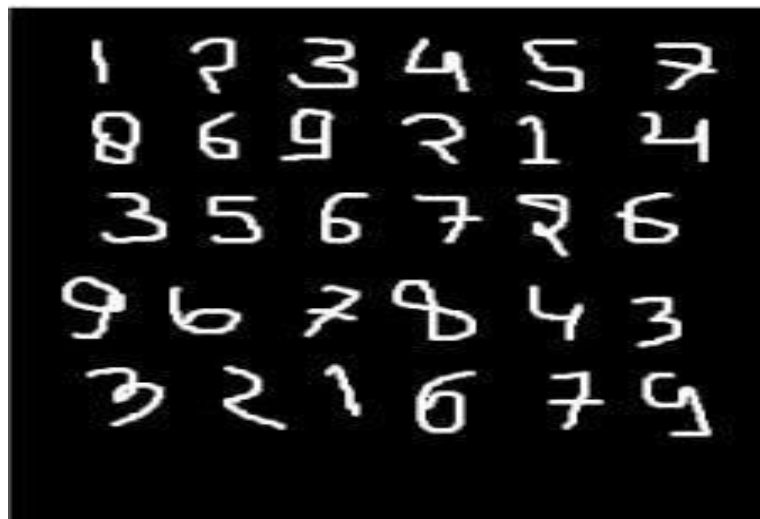
Applying a Convolutional Neural Network (CNN) on the MNIST dataset is a popular way to learn about and demonstrate the capabilities of CNNs for image classification tasks. The MNIST dataset consists of 28×28 grayscale images of hand-written digits (0-9), with a training set of 60,000 examples and a test set of 10,000 examples.

Here is a basic approach of applying CNN on the MNIST dataset using Python Programming Language and the Keras Library.

1. Load and preprocess the data: The MNIST dataset can be loaded using the Keras library, and the images can be normalized to have pixel values between 0 and 1.
2. Define the model architecture: The CNN can be constructed using the Keras Sequential API, which allows for easy building of sequential models layer-by-layer. The architecture should typically include convolutional layers, pooling layers, and fully-connected layers.
3. Compile the model: The model needs to be compiled with a loss function, an optimizer, and a metric for evaluation.
4. Train the model: The model can be trained on the training set using the Keras fit() function. It is important to monitor the training accuracy and loss to ensure the model is converging properly.
5. Evaluate the model: The trained model can be evaluated on the test set using the Keras evaluate() function. The evaluation metric typically used for classification tasks is accuracy.

MNIST Dataset:

mnist dataset is a dataset of handwritten images as shown below in the image



We can get 99.06% accuracy by using CNN(Convolutional Neural Network) with a functional model. The reason for using a functional model is to maintain easiness while connecting the layers.

First include all Libraries:

```
import numpy as np
import keras
from keras.datasets import mnist
from keras.models import Model
from keras.layers import Dense, Input
from keras.layers import Conv2D, MaxPooling2D, Dropout, Flatten
from keras import backend as k
```

Create the train data and test data:

Test data : Used for testing the model that how our model has been trained.

Train data: Used to train our model.

```
(x_train, y_train), (x_test, y_test) = mnist.load_data()
```

While proceeding further, `img_rows` and `img_cols` are used as the image dimensions.

In MNIST dataset, it is 28 and 28. We also need to check the data format i.e. 'channels_first' or 'channels_last'.

In CNN, we can normalize data before hands such that large terms of the calculations can be reduced to smaller terms. Like, we can normalize the `x_train` and `x_test` data by dividing it by 255.

Checking data-format:

```
img_rows, img_cols = 28, 28
if
k.image_data_format()=='channels_first':
x_train=x_train.reshape(x_train.shape[0], 1, img_rows, img_cols)
x_test=x_test.reshape(x_test.shape[0], 1, img_rows, img_cols)
inpx=(1, img_rows, img_cols)
else:
x_train=x_train.reshape(x_train.shape[0], img_rows, img_cols, 1)
x_test=x_test.reshape(x_test.shape[0], img_rows, img_cols, 1)
inpx=(img_rows, img_cols, 1)
x_train=x_train.astype('float32') x_test=x_test.astype('float32') x_train/=255
x_test /=255
```

Description of the output classes:

Since the output of the model can comprise any of the digits between 0 to 9. so, we need 10 classes in output. To make output for 10 classes, use `keras.utils.to_categorical` function, which will provide the 10 columns. Out of these 10 columns, only one value will be one and the rest 9 will be zero and this one value of the output will denote the class of the digit

```
y_train keras.utils.to_categorical(y_train) y_test keras.utils.to_categorical(y_test)
```

Now, the dataset is ready so let's move towards the CNN model :

```
inpx = Input(shape inpx)
layer1=Conv2D(32, kernel_size (3, 3), activation 'relu')(inpx)
layer2=Conv2D(64, (3, 3), activation 'relu')(layer1)
layer3=MaxPooling2D(pool_size (3, 3))(layer2)
layer4 =Dropout(0.5)(layer3)
layer5=Flatten()(layer4)
layer6=Dense(250, activation 'sigmoid')(layer5) layer7 =Dense(10, activation
'softmax')(layer6)
```

Explanation of the working of each layer in the CNN model:

layer1: is the Conv2d layer which convolves the image using 32 filters each of size (3*3).

layer2: is again a Conv2D layer which is also used to convolve the image and is using 64 filters each of size (3*3).

layer3: is the MaxPooling2D layer which picks the max value out of a matrix of size (3*3).

layer4: is showing Dropout at a rate of 0.5.

layer5: is flattening the output obtained from layer4 and this flattens output is passed to layer6.

layer6: is a hidden layer of a neural network containing 250 neurons.

layer7: is the output layer having 10 neurons for 10 classes of output that is using the softmax function.

Calling compile and fit function:

```
model=Model([inpx], layer7) model.compile(optimizer=keras.optimizers.Adadelta(),  
loss=keras.losses.categorical_crossentropy, metrics=['accuracy'])
```

```
model.fit(x_train, y_train, epochs=12, batch_size=500)
```

```
Epoch 1/12  
60000/60000 [=====] - 968s 16ms/step - loss: 0.7357 - acc: 0.7749  
Epoch 2/12  
60000/60000 [=====] - 955s 16ms/step - loss: 0.2087 - acc: 0.9413  
Epoch 3/12  
60000/60000 [=====] - 968s 16ms/step - loss: 0.1287 - acc: 0.9631  
Epoch 4/12  
60000/60000 [=====] - 968s 16ms/step - loss: 0.0948 - acc: 0.9728  
Epoch 5/12  
60000/60000 [=====] - 956s 16ms/step - loss: 0.0780 - acc: 0.9774  
Epoch 6/12  
60000/60000 [=====] - 915s 15ms/step - loss: 0.0655 - acc: 0.9807  
Epoch 7/12  
60000/60000 [=====] - 907s 15ms/step - loss: 0.0575 - acc: 0.9829  
Epoch 8/12  
60000/60000 [=====] - 914s 15ms/step - loss: 0.0498 - acc: 0.9852  
Epoch 9/12  
60000/60000 [=====] - 917s 15ms/step - loss: 0.0468 - acc: 0.9861  
Epoch 10/12  
60000/60000 [=====] - 912s 15ms/step - loss: 0.0420 - acc: 0.9873  
Epoch 11/12  
60000/60000 [=====] - 967s 16ms/step - loss: 0.0405 - acc: 0.9880  
Epoch 12/12  
60000/60000 [=====] - 993s 17ms/step - loss: 0.0371 - acc: 0.9888  
<keras.callbacks.History at 0x21ce04bb6a0>
```

Firstly, we made an object of the model as shown in the above-given lines, where [inpx] is the input in the model and layer7 is the output of the model. We compiled the model using the required optimizer, loss function and printed the accuracy and at the last model.fit was called along with parameters like x_train(means image vectors), y_train(means the label), number of epochs, and the batch size. Using fit function x_train, y_train dataset is fed to model in particular batch size.

Evaluate function:

model.evaluate provides the score for the test data i.e. provided the test data to the model. Now, the model will predict the class of the data, and the predicted class will be matched with the y_test label to give us the accuracy.

```
score = model.evaluate(x_test, y_test, verbose=0) ('loss=', score[0])  
print('accuracy=', score[1])
```

Output:

```
loss= 0.0295960184669
```

```
accuracy = 0.991
```

Algorithm:

1. Import Libraries:

- Import necessary libraries, including TensorFlow and Keras.

2. Load and Pre-Process MNIST Dataset:

- Load the MNIST dataset, which consists of 28x28 grayscale images of handwritten digits (0 through 9)
- Pre-process the data by normalizing pixel values (between 0 and 1), reshaping images, and one-hot encoding labels.

3. Define CNN Model Architecture:

- Design the CNN architecture with convolutional layers, pooling layers, and fully connected layers.
- Use activation functions like ReLU to introduce non-linearity.
- The final layer has 10 units with softmax activation for multiclass classification.

4. Compile the Model:

- Specify the optimizer (e.g., 'adam'), loss function (e.g., 'categorical_crossentropy' for multiclass classification), and evaluation metric (e.g., 'accuracy').

5. Train the Model:

- Train the CNN using the training dataset.
- Specify the number of epochs (passes through the entire dataset) and batch size.

6. Evaluate the Model:

- Evaluate the trained CNN on the test dataset to assess its performance.
- Measure metrics such as accuracy to understand how well the model generalizes to unseen data.

These steps provide a comprehensive overview of the process involved in building and training a CNN for image classification using the MNIST dataset. Adjustments to these steps can be made based on specific model requirements and task objectives.

Application:

1. **Handwritten Digit Recognition:** Recognizes handwritten digits (0-9) with applications in automated systems.
2. **Automatic Check Processing:** Processes checks by recognizing handwritten amounts and account numbers.
3. **Postal Code Recognition:** Recognizes postal codes on envelopes for automated mail sorting.
4. **Document Classification:** Classifies documents based on handwritten patterns or characters.
5. **Medical Imaging:** Analyzes medical images for detecting anomalies or identifying patterns.
6. **Gesture Recognition:** Recognizes hand gestures for sign language translation or device control

Conclusion: Thus I Build a Multiclass classifier using the CNN model using MNIST or any other suitable dataset by Performing Data Pre-processing, Defining Model and perform training and Evaluating Results using confusion matrix.